

SIMATS ENGINEERING

ASSESSMENT TOOL 2 — ASSIGNMENT SUBMISSION

Field	Details
Course Name	Artificial Intelligence
Course Code	CSA17
Course Outcome	CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively (BL3)
Annexure	A — Constraint-Based Problem Solving
Assessment Weightage	35%
Total Marks / Duration	30 Marks / 1 Hour
Student Name / Reg. No.	____192321143_____

Code of Conduct

I _Senthamizhselvan K 192321143_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____ *Senthamizhselvan K* _____

Contents

- Question 1 — Doctor Shift Scheduling (Backtracking Search)
- Question 2 — Grid Navigation Robot (BFS, with A* comparison)
- Question 3 — Autonomous Rescue Robot (Online Search Agent)
- Conclusion

Question 1: Hospital Doctor–Shift Assignment (Backtracking Search)

1.1 Problem Understanding and CSP Formulation

The task is a Constraint Satisfaction Problem (CSP) defined by variables, domains and constraints. Shifts are numbered by time order: Morning (M) = 1, Afternoon (A) = 2, Night (N) = 3.

Component	Definition
Variables	D1, D2, D3 (one variable per doctor)
Domain	{Morning, Afternoon, Night} for each variable
C1	$D1 \neq \text{Night}$
C2	$\text{shift}(D2) < \text{shift}(D3)$ (D2 works before D3)
C3	All-different: no two doctors share a shift (one doctor per shift)
C4	$D3 \neq \text{Morning}$
C5	Every doctor receives exactly one shift (complete assignment)

Constraint analysis: C1 and C4 are unary constraints and can prune domains in advance: $D1 \in \{M, A\}$ and $D3 \in \{A, N\}$. C2 is a binary ordering constraint; because M is the earliest shift, C2 already implies $D3 \neq \text{Morning}$, so C4 is redundant but is still checked. C3 is a global all-different constraint. C5 is the goal test (all three variables assigned).

1.2 Backtracking Search Procedure

Backtracking search is depth-first search over partial assignments. At each step one unassigned variable is chosen, its values are tried in order, each value is checked against all constraints involving already-assigned variables, and the search backtracks as soon as a constraint is violated.

Variable order: $D1 \rightarrow D2 \rightarrow D3$. **Value order:** Morning $\rightarrow$ Afternoon $\rightarrow$ Night.

Step	Assignment tried	Constraint checks	Result
1	D1 = Morning	C1: Morning $\neq$ Night $\checkmark$; C3: no clash $\checkmark$	Accept
2	D2 = Morning (D1=M)	C3: Morning already used by D1 $\times$	Reject $\rightarrow$ next value
3	D2 = Afternoon (D1=M)	C3: Afternoon free $\checkmark$; C1, C4 not applicable to D2; C2 pending until D3 is assigned	Accept
4	D3 = Morning (D1=M, D2=A)	C3: Morning used by D1 $\times$; C4: D3 $\neq$ Morning $\times$; C2: $2 < 1$ false $\times$	Reject $\rightarrow$ next value
5	D3 = Afternoon (D1=M, D2=A)	C3: Afternoon used by D2 $\times$; C2: $2 < 2$ false $\times$	Reject $\rightarrow$ next value
6	D3 = Night (D1=M, D2=A)	C3 $\checkmark$; C4: Night $\neq$ Morning $\checkmark$; C2: $2 < 3$ $\checkmark$; C1 $\checkmark$ (D1 is not Night); C5: all assigned $\checkmark$	Accept $\rightarrow$ GOAL

1.3 Search Tree (depth-first order)

```

Root (empty assignment)
├─ D1 = M ✓
│   └─ D2 = M ✗ (C3 violated)
│       └─ D2 = A ✓
│           └─ D3 = M ✗ (C2, C3, C4 violated)
│               └─ D3 = A ✗ (C2, C3 violated)
│                   └─ D3 = N ✓ → SOLUTION FOUND

```

1.4 Final Valid Schedule

Shift	Doctor Assigned
Morning	D1
Afternoon	D2
Night	D3

Verification: C1 — D1 works Morning, not Night ✓. C2 — D2 (Afternoon) works before D3 (Night) ✓. C3 — each shift has exactly one doctor ✓. C4 — D3 is not on Morning ✓. C5 — all three doctors have exactly one shift ✓.

Remark: Only one backtrack occurred at the value level (steps 2, 4, 5); no earlier variable had to be reassigned. If the search were continued for all solutions, it would find one more valid schedule — D1 = Afternoon, D2 = Morning, D3 = Night — while D1 = Night is pruned immediately by C1. Backtracking returns the first solution found, which is the schedule above.

Question 2: Robot Navigation in a 5×5 Grid (BFS)

2.1 Problem Understanding

The question does not include the grid figure, so a representative 5×5 grid is assumed. Cells are written as (row, column) with the top-left cell as (0,0). **Start S = (0,0)**, **Goal G = (4,4)**, and the **obstacles (X)** are (1,1), (1,2), (1,3), (3,1), (3,2) and (3,3).

	c0	c1	c2	c3	c4
r0	S				
r1		X	X	X	
r2					
r3		X	X	X	
r4					G

Figure 1: Assumed 5×5 environment (S = start, G = goal, X = obstacle)

Component	Definition
State	Robot position (r, c)
Initial state	(0,0)
Goal test	State = (4,4)
Actions	Up, Down, Left, Right (expanded in this order)
Transition	Move one cell; illegal if outside the grid or onto an obstacle
Step cost	1 per move
Heuristic	$h(n) = r - 4 + c - 4 $ (Manhattan distance to G)

Constraint analysis: Obstacles act as hard constraints that remove states from the search space; the four-direction rule gives a branching factor of at most 4; and the uniform step cost of 1 means the path cost equals the number of moves. Because all step costs are equal, Breadth-First Search (BFS) is guaranteed to return an optimal path.

Note on the question: Plain BFS uses a FIFO queue and ignores heuristics, whereas a priority queue ordered by $h(n)$ or $f(n) = g(n) + h(n)$ belongs to Greedy Best-First Search or A*. To satisfy both parts of the question, the main trace below is BFS (FIFO queue) with g and h displayed for reference, and Section 2.4 shows the priority-queue version (A*) for comparison.

2.2 BFS Step-by-Step Node Expansion

Each expansion removes the head of the FIFO queue, generates unvisited neighbours (Up, Down, Left, Right), and appends them to the tail. Columns g / h show path cost and Manhattan distance of the expanded node.

Step	Node expanded	g / h	New nodes added	Queue after expansion
1	(0,0)	0 / 8	(1,0), (0,1)	(1,0), (0,1)
2	(1,0)	1 / 7	(2,0)	(0,1), (2,0)
3	(0,1)	1 / 7	(0,2)	(2,0), (0,2)
4	(2,0)	2 / 6	(3,0), (2,1)	(0,2), (3,0), (2,1)
5	(0,2)	2 / 6	(0,3)	(3,0), (2,1), (0,3)
6	(3,0)	3 / 5	(4,0)	(2,1), (0,3), (4,0)
7	(2,1)	3 / 5	(2,2)	(0,3), (4,0), (2,2)
8	(0,3)	3 / 5	(0,4)	(4,0), (2,2), (0,4)
9	(4,0)	4 / 4	(4,1)	(2,2), (0,4), (4,1)
10	(2,2)	4 / 4	(2,3)	(0,4), (4,1), (2,3)
11	(0,4)	4 / 4	(1,4)	(4,1), (2,3), (1,4)
12	(4,1)	5 / 3	(4,2)	(2,3), (1,4), (4,2)
13	(2,3)	5 / 3	(2,4)	(1,4), (4,2), (2,4)
14	(1,4)	5 / 3	—	(4,2), (2,4)
15	(4,2)	6 / 2	(4,3)	(2,4), (4,3)
16	(2,4)	6 / 2	(3,4)	(4,3), (3,4)
17	(4,3)	7 / 1	(4,4)	(3,4), (4,4)
18	(3,4)	7 / 1	—	(4,4)
19	(4,4)	8 / 0	—	Empty (goal reached)

The goal (4,4) is reached at step 19 with $g = 8$. A total of **19 nodes** were expanded.

2.3 Optimal Path and Cost

Following parent pointers back from G gives the path:

$(0,0) \rightarrow (1,0) \rightarrow (2,0) \rightarrow (3,0) \rightarrow (4,0) \rightarrow (4,1) \rightarrow (4,2) \rightarrow (4,3) \rightarrow (4,4)$

	c0	c1	c2	c3	c4
r0	S				
r1	•	X	X	X	
r2	•				
r3	•	X	X	X	
r4	•	•	•	•	G

Figure 2: Optimal path found by BFS (marked •)

Path cost = 8 (8 moves $\times$ cost 1). Moves: Down, Down, Down, Down, Right, Right, Right, Right.

Optimality check: The Manhattan distance from S to G is $4 + 4 = 8$, which is a lower bound on any path cost. Since the path found costs exactly 8, no cheaper path can exist. BFS is also complete and optimal here because all step costs are equal. Other paths of cost 8 exist (for example along the top row and right column); BFS returns the first one it finds under the chosen action order.

2.4 Priority-Queue Version (A*) for Comparison

Using a priority queue ordered by $f(n) = g(n) + h(n)$, with ties broken by smaller h (the node nearer the goal), gives the following trace. Columns $g / h / f$ show the expanded node's values; the last column lists the open list with f -values.

Step	Node expanded	$g / h / f$	Inserted (f -value)	Priority queue after step
1	(0,0)	0 / 8 / 8	(1,0)[$f=8$], (0,1)[$f=8$]	(0,1)[8], (1,0)[8]
2	(0,1)	1 / 7 / 8	(0,2)[$f=8$]	(0,2)[8], (1,0)[8]
3	(0,2)	2 / 6 / 8	(0,3)[$f=8$]	(0,3)[8], (1,0)[8]
4	(0,3)	3 / 5 / 8	(0,4)[$f=8$]	(0,4)[8], (1,0)[8]
5	(0,4)	4 / 4 / 8	(1,4)[$f=8$]	(1,4)[8], (1,0)[8]
6	(1,4)	5 / 3 / 8	(2,4)[$f=8$]	(2,4)[8], (1,0)[8]
7	(2,4)	6 / 2 / 8	(3,4)[$f=8$], (2,3)[$f=10$]	(3,4)[8], (1,0)[8], (2,3)[10]
8	(3,4)	7 / 1 / 8	(4,4)[$f=8$]	(4,4)[8], (1,0)[8], (2,3)[10]
9	(4,4)	8 / 0 / 8	—	(1,0)[8], (2,3)[10]

A* reaches the goal with the same optimal cost of **8** after only **9 expansions** (path $(0,0) \rightarrow (0,1) \rightarrow (0,2) \rightarrow (0,3) \rightarrow (0,4) \rightarrow (1,4) \rightarrow (2,4) \rightarrow (3,4) \rightarrow (4,4)$), compared with 19 for BFS. The Manhattan heuristic is admissible and consistent on a 4-connected grid with unit costs, so A* is optimal while expanding far fewer nodes.

Criterion	BFS	A* (Manhattan)
Queue type	FIFO queue	Priority queue on $f = g + h$
Uses heuristic	No	Yes
Nodes expanded	19	9
Path cost	8 (optimal)	8 (optimal)

Question 3: Autonomous Rescue Robot in a Disaster Building

3.0 Problem Understanding

A rescue robot starts at S in a grid of rooms and must reach a survivor at G. The map is only partially known: some cells are blocked, some are risky, and the robot can sense only its four adjacent cells. The robot must therefore plan, act, sense and re-plan while minimising total cost and avoiding unsafe moves.

(a) Analysis of Constraints and Their Effect on Decision-Making

Constraint	Type	Effect on the robot's decisions
Moves: up, down, left, right	Action constraint	Limits branching factor to 4; the shortest-distance lower bound is the Manhattan distance.
Blocked cells (obstacles)	Hard constraint	Blocked cells are removed from the search space; plans must route around them, and discovering one can invalidate the current plan.
Sensing only adjacent cells	Information constraint	The environment is partially observable and unknown in advance, so the robot cannot compute a full plan once. It must keep an internal map, update it after every move and re-plan.
Move cost 1; risky cell adds 2 (total 3)	Cost / soft constraint	Path cost is no longer the number of steps, so BFS is unsuitable. A risky cell is worth entering only if the detour around it costs more than 2 extra moves.
Minimise total cost and navigate safely	Optimisation + safety	Cost-aware search is needed. Safety is enforced by never moving into a cell known to be blocked and by penalising risky cells.
Dynamic knowledge update (online search)	Interaction constraint	Planning and execution are interleaved: sense → update map → plan → move → repeat.

Prioritisation of constraints: (1) Hard constraints come first — never enter a blocked cell and stay inside the grid. (2) Safety next — avoid risky cells unless the saving justifies the penalty. (3) Cost minimisation follows — choose the lowest-cost route among the safe options. (4) Information handling runs throughout — treat unknown cells optimistically but re-plan when they turn out to be blocked or risky.

(b) Solution Approach: Online Uniform-Cost Search with Re-planning

The chosen strategy is an **Online Search Agent that uses Uniform Cost Search (UCS) as its planner**. The robot keeps a map of known cells and re-runs UCS from its current position each time new information is sensed.

Cost function: the cost of entering a cell is 1 for a normal cell and $1 + 2 = 3$ for a risky cell. $g(n)$ is the sum of these costs along the path, and UCS always expands the frontier node with the lowest $g(n)$.

Steps of the algorithm:

Step 1: **Initialise knowledge:** record S and G; mark all other cells as unknown. Unknown cells are assumed to be free with cost 1 (the optimistic "free-space assumption").

- Step 2: **Sense:** at the current cell, observe the four adjacent cells and label each as free, risky or blocked. Update the internal map.
- Step 3: **Goal test:** if the current cell is G, the survivor is reached — stop.
- Step 4: **Plan:** run UCS on the current map from the current cell to G. Blocked cells are excluded; risky cells cost 3; unknown cells cost 1.
- Step 5: **Act:** move one step along the cheapest path, only into a cell that has been sensed as non-blocked (this guarantees the robot never bumps into an obstacle).
- Step 6: **Re-plan:** after the move, sense again. If the map has changed (new obstacle, new risky cell), the old plan may no longer be cheapest, so run UCS again.
- Step 7: **Dead end handling:** if UCS finds no path, the robot backtracks along its visited cells to the nearest frontier with unexplored neighbours and resumes. If no frontier remains, G is unreachable.

Pseudocode

```
function ONLINE-UCS-AGENT(percept):
    update MAP with percept (adjacent cells: free / risky / blocked)
    if position == GOAL: return STOP
    plan <- UCS(MAP, position, GOAL)          # cost: 1 normal, 3 risky
    if plan is empty: plan <- BACKTRACK-TO-FRONTIER(MAP)
    return first action of plan                # then sense again, repeat
```

Illustration: Suppose the cheapest known route to G has 5 moves and passes through one risky cell, costing $5 + 2 = 7$, while a safe detour has 6 moves costing 6. UCS selects the safe detour ($6 < 7$). If the robot later senses that the detour is blocked, it re-plans and takes the risky route at cost 7, as this is now the cheapest available option. This shows how risk, cost and dynamic knowledge interact.

(c) Application of AI Concepts

1. Intelligent agent. An agent perceives its environment through sensors and acts on it through actuators. The rescue robot is a **model-based, utility-oriented (goal-based) agent**: it maintains an internal map (the model), has a goal (reach the survivor), and compares actions by cost (utility).

PEAS element	Rescue robot
Performance measure	Reach survivor; minimise total path cost (moves + risk penalty); avoid blocked and unsafe cells
Environment	Grid of rooms in a damaged building with free, blocked and risky cells
Actuators	Movement: Up, Down, Left, Right
Sensors	Adjacent-cell sensors (blocked / free / risky) and current position

2. Rationality. A rational agent selects, for every percept sequence, the action expected to maximise its performance measure given its knowledge. Here, the robot chooses the move on the lowest-cost known path to G, avoids risky cells when a cheaper safe alternative exists, and

updates its choices as soon as new percepts arrive. Because the robot has limited information, its rationality is relative to what it knows. It is optimal with respect to its current map, not with respect to the true map it has not yet seen.

3. Environment type.

Property	Classification	Reason
Observability	Partially observable	Sensors reveal only adjacent cells
Agents	Single agent	Only the rescue robot acts (survivor is stationary)
Determinism	Deterministic (assumed)	A chosen move has a predictable result; aftershocks would make it stochastic
Episodic vs sequential	Sequential	Each move affects future positions and costs
Static vs dynamic	Static / semi-dynamic	The map changes only through the robot's discovery (collapses would make it dynamic)
Discrete vs continuous	Discrete	Grid cells and four discrete actions
Known vs unknown	Unknown (initially)	Map must be learned by exploration — hence online search

(d) Justification of the Chosen Approach

- **Handles unequal costs:** UCS minimises the actual path cost including the risk penalty. BFS and DFS ignore costs, and BFS is optimal only when all steps cost the same.
- **Handles partial observability:** an offline planner would need the full map in advance. The online agent senses, updates its knowledge and re-plans, which is exactly what a limited-sensor robot needs.
- **Optimal on known information and complete:** UCS returns the cheapest path on the current map and never gets stuck in loops. With backtracking to frontiers, the robot will find G whenever G is reachable and the environment does not contain irreversible traps.
- **Safe by design:** the robot moves only into cells it has already sensed as non-blocked, and risky cells carry a penalty, so safety and cost are balanced in one framework.
- **Simple and uses no heuristic:** UCS needs no domain heuristic, so it stays correct with unknown cells and varying costs. An admissible Manhattan heuristic can be added later (online A*) to expand fewer nodes, as shown in Question 2.

Strategy	Handles costs	Handles unknown map	Suitability
BFS	No (equal costs only)	No	Poor — ignores risk penalty
DFS	No	No	Poor — not optimal, may wander
Greedy best-first	No	Partly	Poor — ignores accumulated cost, not optimal
Offline UCS / A*	Yes	No	Good if the map is known; not applicable here

Strategy	Handles costs	Handles unknown map	Suitability
Online UCS with re-planning	Yes	Yes	Best fit for this problem

Limitation: because knowledge is incomplete, no online algorithm can guarantee the globally cheapest route in the real building; the path is the best possible given what has been observed so far, and the cost depends on how much re-planning is needed.

Conclusion

Question	Technique	Result
1	Backtracking search (CSP)	D1 = Morning, D2 = Afternoon, D3 = Night
2	BFS (with A* comparison)	Path (0,0)→(1,0)→(2,0)→(3,0)→(4,0)→(4,1)→(4,2)→(4,3)→(4,4), cost 8
3	Online UCS agent with re-planning	Cost-aware, safe, adaptive navigation under partial observability